

Laboratory protocol

Dice

Kotschnig Thomas, Eingang Fabian

Table of Content

1	Task description	1
2	Implementation	2
2.1	Block diagram	2
2.2	Waveform shreenshots	3
2.3	RTL Code	5
2.4	Photos of the FPGA board	7
3	Date and Signature	9

1 Task description

In this laboratory exercise, several dices were implemented on the BASYS3 FPGA board. In the task description a compile able SystemVerilog project was provided. In this project it was necessary to implement a pseudo-random functionality to simulate a “fair” dice. In the project, up to four independent dice can be enabled via switches. One additional switch has been implemented to reset the system with an active low reset signal and a button on the FPGA-board is used to simulate a “throw” of the activated dices. The implemented design was first verified using the provided testbench and then synthesized and tested on the physical hardware.

2 Implementation

2.1 Block diagram

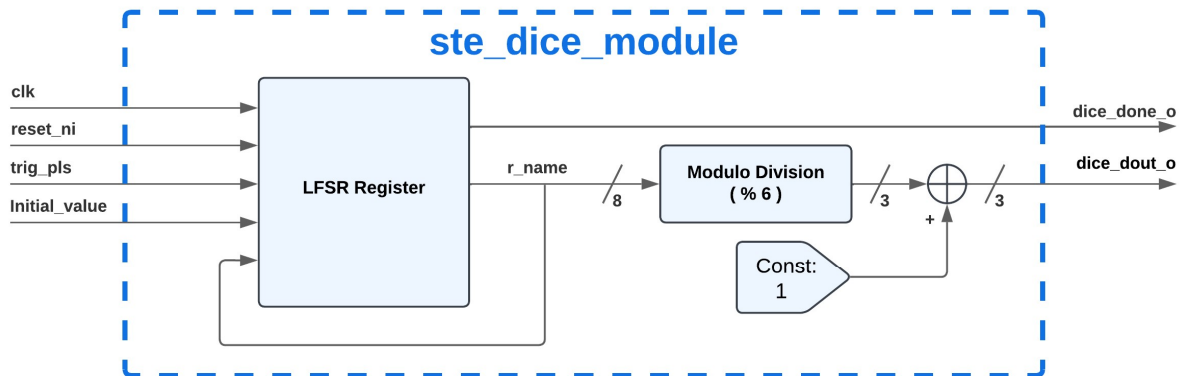


Figure 1: Block diagram of the dice module implementation

As it can be seen in Figure 1 our dice module consists of an LFSR, a modulo division block and an incrementation. The LFSR generates the pseudo random values for our dice result. The register is loaded at initialisation and at reset with an initial value. Those initial values are different for each dice so that the dices have different results at each throw.

The register value output of the LFSR is an 8-bit signal. This signal can therefore be between 0 and 255. To map it down to the values which are valid for a dice a modulo division is done. By modulo dividing the register values through 6, the range for possible output values changes from 0-255 to 0-5. To receive now valid dice values this 3-bit output signal of the modulo division is incremented by one. Output range for the dice values: 1-6.

2.2 Waveform shreenshots

In the testbench two dices have been simulated. Both dices got thrown 1000 times to see how the result distributed. Our *dice_val_cnt* variable tracks the results of one of the dices.

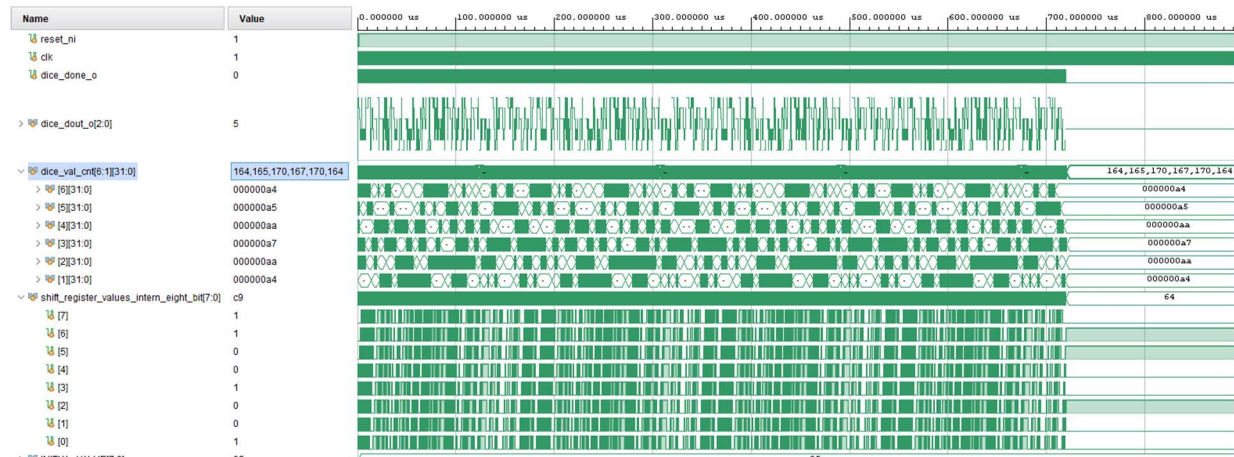


Figure 2: Testbench simulating 1000 dice throws

As it can be seen out of the simulation the implemented LFSR shift register implements a fair dice. The distribution how frequent the dice result appeared can be seen in the following Table 1. Each possible dice result appeared between 164 and 170 times.

Table 1: Dice results after 1000 simulated throws

Dice result	Number of throws
1	164
2	170
3	167
4	170
5	165
6	164
Sum	1000

2 Implementation

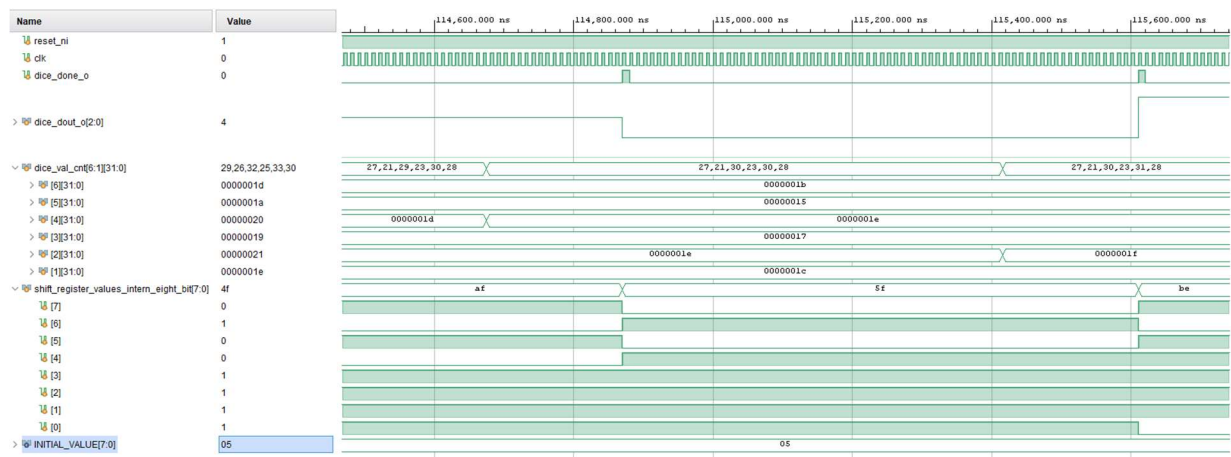


Figure 3: Simulation of one single dice throw

Figure 3 shows the important signals when simulating one single throw from the dice. The signal *dice_done_o* signals the calculation for a dice throw has been finished. This signal is a pulse which is high for one clock period.

2.3 RTL Code

For a better readability the variable name *shift_register_values_intern_eight_bit* which is used in our SystemVerilog simulations and sourcecode has in the following RTL code been changed to *r_name*.

```

                                ste_dice_module.sv
1. `default_nettype none
2. module ste_dice_module #(
3.     parameter bit[7:0] INITIAL_VALUE = 8'b0100_00001
4. ) (
5.     input  wire clk          , // I; System clock
6.     input  wire reset_ni     , // I; system cock reset (active low)

7.     //Push Button Inputs
8.     input  wire trig_pls     , // I 1; Center
9.     // Seven Segment Display Outputs
10.    output logic dice_done_o   , // O 1; Dice rolling done
11.    output logic [2:0] dice_dout_o // O 3; Dice result
12. );
13. // -----
14. // Definition
15. // -----
16. logic [7:0] r_name;

17. // -----
18. // Implementation
19. // -----
20. always_ff @(posedge clk or negedge reset_ni)
21. begin
22.     if(~reset_ni)
23.     begin
24.         r_name <= INITIAL_VALUE;
25.         dice_done_o <= 1'b0;
26.         dice_dout_o <= 3'b000;
27.     end
28.     else if(trig_pls)
29.     begin
30.         r_name <= {r_name[6:0], (r_name[7] ^ r_name[5] ^ r_name[4] ^
r_name[3])};
31.         dice_done_o <= 1'b1;
32.         dice_dout_o <= r_name % 6 + 1;
33.     end
34.     else
35.     begin
36.         dice_done_o <= 1'b0;
37.     end
38. end
39. endmodule
40. `default_nettype wire

```

2 Implementation

The RTL code shows how we implemented the pseudo random number generator. The 8-bit register is loaded with an initial value, which was chosen different for each of the dices. This value gets loaded into the register at every reset that is applied (Line 24)

To fit the requirements of the task description the dice can be reset with an active low signal (Line 22).

The *trig_pls* signal which starts the calculation for each throw is an input signal of our dice module and comes from the *btnC* signal in the testbench (Line 8 is the input definition and in Line 28 it is used to start the calculation). In the implementation the "BTNC <U18>" button on the FPGA board can create the high pulse for rolling the dices.

The calculation of the pseudo random register value of the LFSR register is conducted in Line 30.

To create valid dice values out of the 8 bit register (value range 0-255) the register value is modulo divided through 6. This division results in values from 0 to 5. Therefore the result is then incremented by one. This calculation can be seen in Line 32.

2.4 Photos of the FPGA board

The program was tested on a BASYS3 Board to verify the code a real environment. In the Figure 4 the reset Button was tested which resulted in showing four zeroes on the 7-segment display.

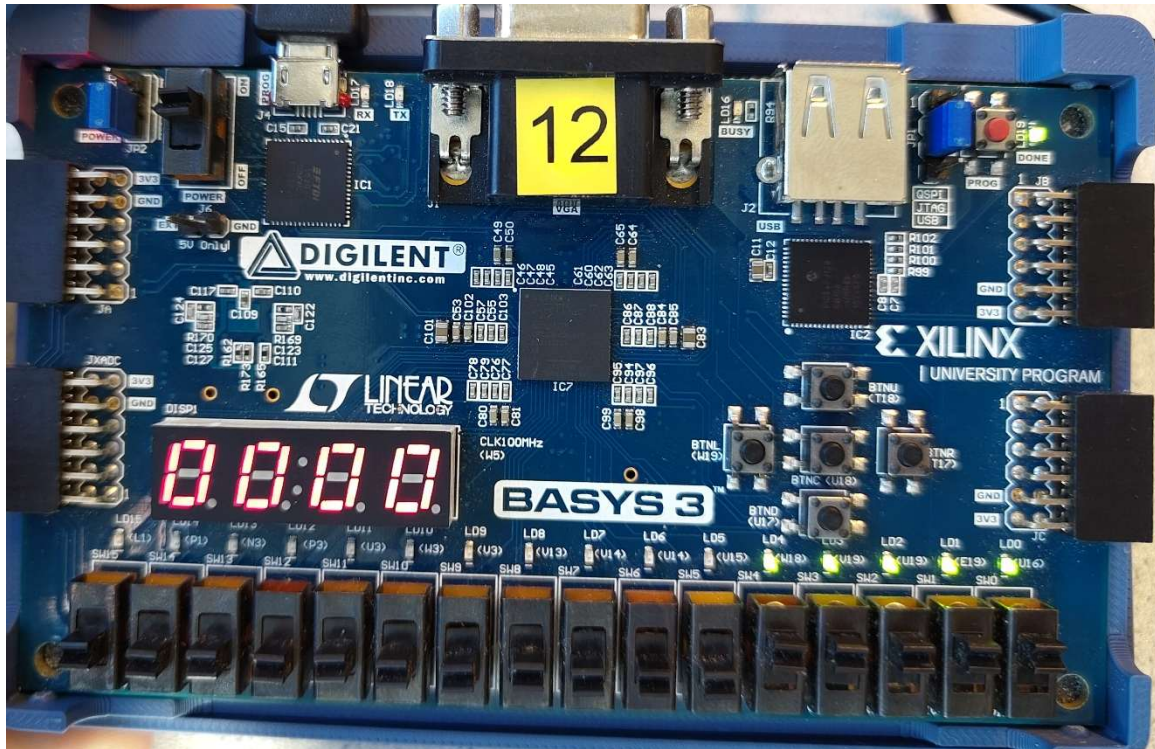


Figure 4: BASYS 3 FPGA Board after pressing the reset button.

2 Implementation

In the Figures 5 & 6 a simulated dice roll can be seen. The selection of number of dices was tested in this scenarios as well. In the case of less than four selected dices, the display lights the dot of the 7-segment display in this position instead.

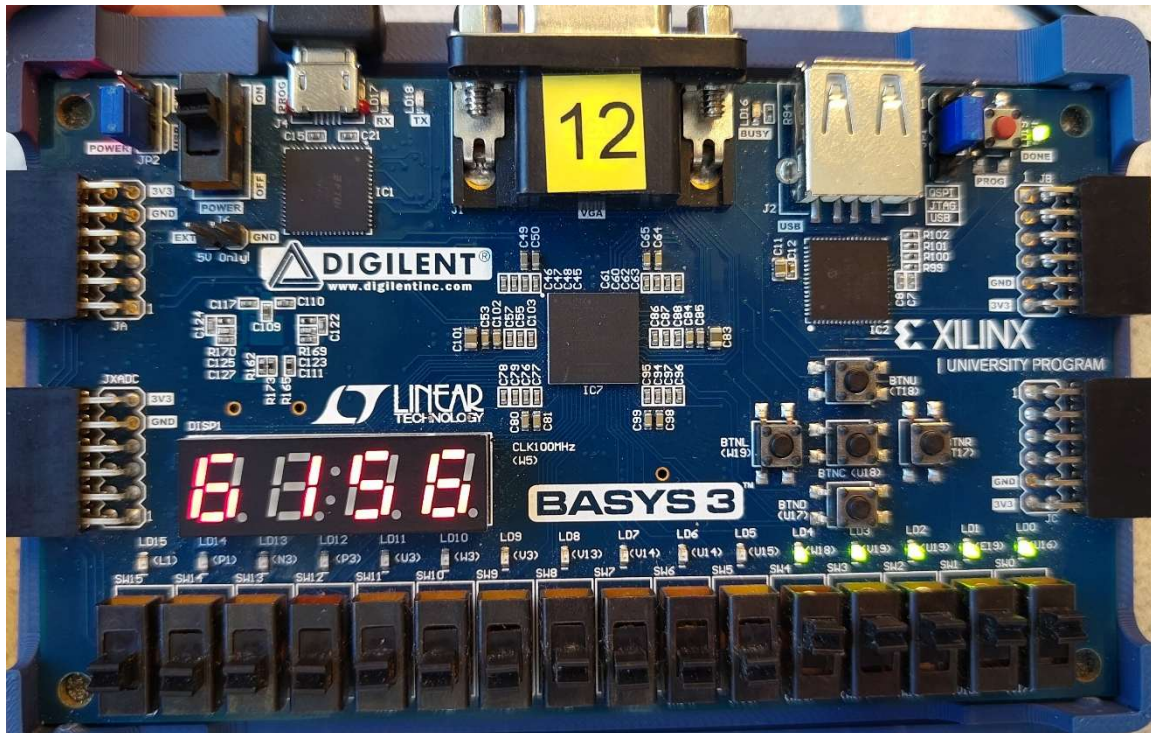


Figure 6: BASYS 3 FPGA Board after pressing dice roll button with 4 activated dices.

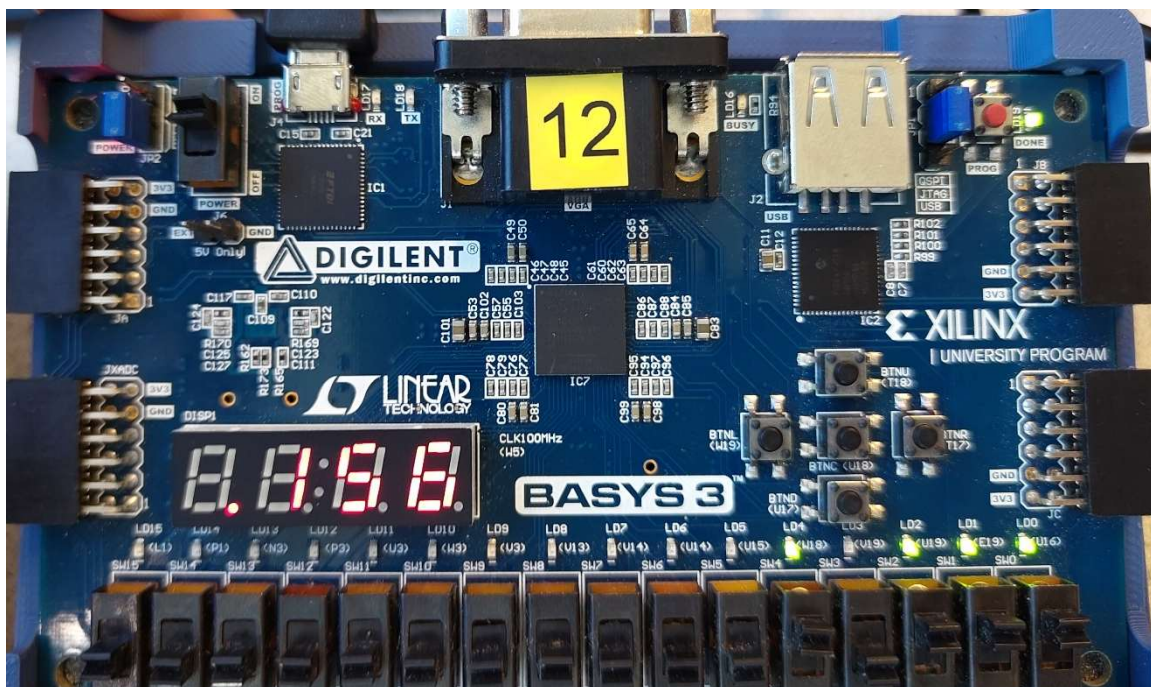


Figure 5: BASYS 3 BASYS 3 FPGA Board after pressing dice roll button with 3 activated dices.

3 Date and Signature

Graz, at 08.12.2025



Eingang Fabian BSc.



Kotschnig Thomas BSc.